

MACHINE LEARNING ENGINEERING

FASE 5 | AULA 01 -

POR QUE CONTROLAR DADOS E MODELOS?

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
MERCADO, CASES E TENDÊNCIAS	14
O QUE VOCÊ VIU NESTA AULA?	17
REFERÊNCIAS	18

EMSE

O QUE VEM POR AÍ?

Imagine implantar um sistema de inteligência artificial em produção sem controle das versões de dados e modelos. Como garantir que os resultados de ontem seriam os mesmos se o modelo fosse reexecutado hoje? Nesta aula, exploraremos os fundamentos do gerenciamento de dados e modelos no contexto de projetos de Machine Learning. Esse tema é central em Machine Learning Engineering, pois diferentemente do desenvolvimento tradicional de software, sistemas de ML apresentam comportamento estocástico, dependência intensa da qualidade dos dados e estão sujeitos a constante evolução conforme novos dados surgem e modelos são retrabalhados.

O controle sistemático de versões de dados e modelos tornou-se fundamental para garantir a reprodutibilidade dos experimentos, a rastreabilidade das decisões algorítmicas, a conformidade com exigências legais e a evolução sustentável de soluções de ML em produção. Por exemplo, Sculley et al. (2015) demonstraram que a falta de práticas rigorosas de controle de artefatos de ML gera um acúmulo de dívida técnica que compromete a manutenibilidade e confiabilidade de sistemas inteligentes.

Ao longo desta aula, apresentaremos conceitos-chave como versionamento de datasets, registro de experimentos, gerência de artefatos de modelos e monitoramento de data drift (deriva de dados). Esses tópicos serão embasados em fundamentos teóricos, equações e estatísticas relevantes, práticas de engenharia consagradas pela comunidade de MLOps e estudos de caso da literatura. Ao final, você compreenderá a importância crítica do gerenciamento sistemático de dados e modelos, identificará riscos da falta de controle adequado e estará apto(a) a aplicar práticas fundamentais de MLOps em projetos reais de Machine Learning.

HANDS ON

Nesta seção prática, vamos demonstrar o uso do Data Version Control (DVC) em um workflow completo de versionamento de dados. As videoaulas correspondentes ao hands on guiarão você passo a passo na implementação de um sistema de controle de versões para datasets e modelos, destacando os principais desafios e soluções na prática. Utilizaremos um ambiente de desenvolvimento configurado com Git e DVC, simulando cenários reais de engenharia de ML. Certifique-se de ter o Python 3.8+ instalado, bem como o DVC (versão 3.0 ou superior) e acesso a uma conta no GitHub para clonar o repositório com o código completo.

Código completo disponível no GitHub.

Repositório: [FIAP-DATA-VERSION-CONTROL-ML-FLOW](https://github.com/CIAP-DATA-VERSION-CONTROL-ML-FLOW)

Exemplo de comando: inicialização do DVC e adição de um arquivo de dados.

```
1 git init # Inicializa repositório Git
2 dvc init # Inicializa controle de versão de dados (cria
pasta .dvc)
3 dvc add dados/brutos/dataset.csv # Adiciona dataset
bruto ao DVC
4 git add . # Adiciona metadados (.dvc) e config do DVC
ao Git
5 git commit -m "Versão inicial do dataset com DVC"
```

Código-fonte 1 - Comandos iniciais de configuração e versionamento de dados com DVC (Bash)

Fonte: Elaborado pelo autor (2026)

Após a execução desses passos, teremos um histórico completo tanto do código quanto dos dados utilizados em cada experimento. Assista ao vídeo 3 desta aula para acompanhar a execução completa do pipeline de versionamento de dados e modelos usando o DVC, incluindo a visualização dos arquivos de metadados gerados e a realização de operações de push/pull para um remote local. No vídeo 4, você verá a aplicação completa integrada em um caso prático de detecção de fraude, onde todas essas etapas de versionamento de dados e modelos são utilizadas para treinar, implantar e monitorar um modelo de ML de ponta a ponta.

SAIBA MAIS

Versionamento de código-fonte (por exemplo, usando Git) já é um conceito consolidado em desenvolvimento de software. Entretanto, o versionamento de dados em Machine Learning traz desafios únicos que requerem novas abordagens e ferramentas dedicadas. Nesta seção, examinaremos por que controlar dados e modelos é crítico para projetos de ML, explorando aspectos teóricos, matemáticos e práticos.

Diferenças entre código e dados em ML

Em projetos tradicionais de software, o código-fonte é relativamente pequeno e suas alterações costumam ser bem localizadas e textuais (adição, remoção ou modificação de linhas). Já em Machine Learning, os datasets podem ter tamanhos massivos (de gigabytes a petabytes) e mudar de forma não-linear ao longo do tempo, conforme novos dados são coletados ou processados. Ou seja, dados de treinamento de modelos de ML não são estáticos – eles evoluem continuamente por meio de operações como adição de registros, limpeza de outliers, enriquecimento com novas variáveis ou filtragem de instâncias inválidas.

Cada transformação realizada em um conjunto original gera uma nova versão do dataset. Halevy et al. (2016) descrevem formalmente o histórico de um dataset como uma sequência temporal de versões, em que cada transformação (T_i) aplicada a uma versão gera a próxima: ($D_i = T_i(D_{i-1})$). Assim, podemos representar o histórico de versões de um dataset como:

$$D=\{D_0, D_1, D_2, \dots, D_t\},$$

Onde: cada (D_i) é o estado do dataset no instante (i), e ($D_i = T_i(D_{i-1})$). A figura a seguir ilustra esse conceito, mostrando um dataset inicial (D_0) que é transformado sucessivas vezes (adição de registros e atributos, modificações e correções), gerando novas versões (D_1 , D_2 , D_3), e assim por diante.



Figura 1 – Evolução temporal de um dataset através de sucessivas versões ((D_0 \to D_1 \to D_2 \to D_3)).

Fonte: Elaborado pelo autor (2026).

Figura 1 – Evolução temporal de um Dataset

Fonte: Elaborado pelo autor (2026)

A capacidade de representar e recuperar eficientemente toda a sequência de versões ($\{D_0, \dots, D_t\}$), minimizando redundâncias de armazenamento e mantendo metadados de rastreabilidade, é fundamental para garantir a reprodutibilidade experimental (HALEVY et al., 2016). Nesse contexto, surge a necessidade de sistemas de versionamento de dados especializados, pois as ferramentas de controle de versão de código tradicionais não lidam bem com arquivos de grande porte nem com operações não-lineares de modificação de dados. Vartak et al. (2016) propuseram uma taxonomia formal de requisitos para sistemas de versionamento de datasets em ML, apontando características essenciais como: (a) suporte a grandes volumes de dados, (b) manipulação de múltiplas ramificações e junções (branch/merge) de versões de um mesmo conjunto de dados, (c) diferenciação eficiente entre versões e (d) integração transparente com fluxos de trabalho de ciência de dados.

Complexidade da diferenciação de dados

Um dos principais desafios técnicos do versionamento de dados está na comparação (diff) entre versões volumosas. Enquanto o git utiliza algoritmos de diff baseados em texto (linhas) adequados para código-fonte, esses métodos são inviáveis para grandes arquivos binários de dados. Bhardwaj et al. (2015) demonstraram que algoritmos tradicionais de diff binário apresentam complexidade temporal ($O(n \cdot m)$) ao comparar dois arquivos de tamanhos (n) e (m) – uma

complexidade quadrática que se torna rapidamente proibitiva para datasets na ordem de gigabytes ou mais. Além disso, armazenar integralmente cada versão de um dataset seria extremamente ineficiente: se um dataset de 10 GB for versionado 100 vezes ingenuamente, o armazenamento total necessário (S_{naive}) seria a soma do tamanho de todas as versões:

$$S_{\text{ingênuo}} = \sum_{i=0}^t \|D_i\|,$$

Figura 2 – Armazenamento total necessário
Fonte: Elaborado pelo autor (2026)

Neste exemplo, resultaria em até 1 terabyte para 100 versões de um dataset de 10 GB. Claramente, essa abordagem ingênua é impraticável. Em vez disso, sistemas modernos de versionamento de dados empregam armazenamento por conteúdo (content-addressable storage) e deduplicação para reduzir drasticamente a redundância. Se representarmos por $\Delta(D_{i-1}, D_i)$ as diferenças incrementais entre a versão (D_{i-1}) e (D_i), o espaço total otimizado ($S_{\text{otimizado}}$) pode ser aproximado por:

$$S_{\text{otimizado}} = \|D_0\| + \sum_{i=1}^t \Delta(D_{i-1}, D_i).$$

Figura 3 – Espaço total otimizado
Fonte: Elaborado pelo autor (2026)

Na prática, estudos mostram que essa estratégia de armazenar apenas deltas pode reduzir as necessidades de armazenamento em 85–95% em comparação ao método ingênuo para conjuntos de dados reais (MIAO et al., 2017). Em outras palavras, em vez de ocupar 1 TB para 100 versões, o uso de um sistema de versionamento com deduplicação pode comprimir o histórico completo em algo em torno de 50–150 GB, dependendo da similaridade entre as versões (isto é, do volume de alterações em cada iteração).

Arquitetura técnica do DVC

O Data Version Control (DVC) é uma das ferramentas pioneiras em versionamento de dados e modelos para ML, construída seguindo os princípios já mencionados. Sua arquitetura se baseia em três componentes fundamentais que trabalham em conjunto para implementar o versionamento distribuído de dados: o sistema de metadados, o cache local e o storage remoto.

- **Sistema de metadados:** o DVC armazena metadados de cada arquivo de dados versionado em arquivos especiais com extensão “.dvc”, que são versionados no Git junto com o código do projeto. Cada arquivo .dvc contém informações como: caminho original do arquivo de dados; hash MD5 do conteúdo do arquivo; tamanho em bytes; e o apontamento para a localização do conteúdo real no storage. Esses metadados permitem ao DVC identificar mudanças no dataset (por exemplo, se o arquivo foi modificado, movido ou removido), sem precisar armazenar o próprio conteúdo dos dados no Git.
- **Cache local:** ao adicionar um arquivo ao DVC (dvc add), o sistema calcula um hash criptográfico (por padrão, MD5) do conteúdo do arquivo e armazena esse hash no arquivo .dvc. Em seguida, o DVC move (ou copia) o arquivo para um diretório oculto de cache local (tipicamente localizado em .dvc/cache/), renomeando-o com seu hash como nome. Dessa forma, se outro arquivo ou versão possuir o mesmo conteúdo, o DVC detectará a colisão de hash e evitará duplicação, utilizando a mesma entrada de cache para ambas as versões do dado. Essa abordagem de armazenamento por hash – também conhecida como content-addressable storage – garante integridade e economia de espaço. A chance de colisão de dois arquivos diferentes gerarem o mesmo hash MD5 é extremamente baixa: para (k) arquivos, a probabilidade de colisão ($P_{\text{colisão}}(k)$) é dada aproximadamente por:

$$P_{\text{colisão}}(k) \approx 1 - e^{-\frac{k^2}{2 \cdot 2^{128}}},$$

Figura 4 – Probabilidade de colisão
Fonte: Elaborado pelo autor (2026)

Onde: (2^{128}) é o número de hashes possíveis do MD5. Para se ter ideia, mesmo com ($k = 10^9$) (um bilhão) de arquivos, ($P_{\text{colisão}} < 10^{-19}$) – uma probabilidade desprezível na prática. Assim, podemos confiar que cada versão de arquivo será identificada unicamente pelo seu hash (por exemplo, a3bcb854fa3...) e armazenada no cache local apenas uma vez.

- **Storage remoto:** embora o cache local seja suficiente para uso pessoal, em cenários de colaboração é necessário um storage centralizado (em nuvem ou rede) para compartilhar dados versionados. O DVC integra-se facilmente a diversos sistemas de armazenamento remoto (Amazon S3, Google Cloud Storage, Azure Blob, repositórios SSH etc.). Quando executamos `dvc push`, os arquivos do cache local (identificados por seus hashes) são enviados para o remote configurado. Da mesma forma, colegas de equipe podem usar `dvc pull` para obter os arquivos necessários do remote. Esse modelo de storage compartilhado, aliado aos metadados versionados no Git, possibilita que múltiplos usuários trabalhem no mesmo projeto de ML compartilhando os datasets e modelos de forma eficiente e consistente.

Formalização da reprodutibilidade experimental

Garantir que um experimento de ML possa ser repetido e produzir os mesmos resultados envolve controlar todas as variáveis relevantes. Podemos formalizar um experimento de ML como uma função matemática determinística que recebe quatro componentes principais (Código, Dados, Parâmetros e Seeds aleatórios) e produz um resultado (modelo treinado, métricas etc.):

$$E: (C \times D \times P \times S) \rightarrow R,$$

Onde:

- (C) representa o código executado (implementação do modelo ou algoritmo);
- (D) é o dataset de entrada utilizado no treinamento;
- (P) denota o conjunto de parâmetros e hiperparâmetros do experimento;
- (S) simboliza o estado inicial do gerador de números aleatórios (random seed);
- (R) é o resultado do experimento (ex.: métricas de desempenho e artefatos do modelo treinado).

Dizemos que um experimento é perfeitamente reproduzível se, dados dois conjuntos de entradas idênticos, o experimento produzir exatamente o mesmo resultado. Formalmente: para todos $((c, d, p, s))$ e $((c', d', p', s')) \in C \times D \times P \times S$: se $((c, d, p, s) = (c', d', p', s'))$ então $(E(c, d, p, s) = E(c', d', p', s'))$. Na prática, alcançar essa igualdade perfeita pode ser desafiador devido a fatores não determinísticos em ML (como inicialização aleatória de pesos, rounding numérico, threads concorrentes etc.). Entretanto, ao controlar rigorosamente código, dados e parâmetros, podemos chegar cada vez mais próximo da reprodutibilidade ideal.

Ferramentas como DVC contribuem fornecendo controle de versão para (D) (dados) e integrando-se a sistemas de controle de código (como Git) para versionar (C) e registrar (P). Já a gerência de (S) (semente aleatória) depende de o pesquisador configurar adequadamente fontes de aleatoriedade determinísticas (por exemplo, fixar seeds em bibliotecas de ML durante o treinamento). Em conjunto, essas práticas permitem recriar um experimento passado e obter os mesmos resultados, desde que todos os componentes sejam mantidos iguais.

Métricas de reprodutibilidade

Como avaliar quantitativamente se um experimento é ou não reproduzido com sucesso? Uma proposta interessante é apresentada por Gundersen e Kjensmo (2018), que definem uma métrica (ρ) de diferença entre os resultados de duas execuções do experimento (E_1) e (E_2). Suponha que (M) é o conjunto de métricas de desempenho avaliadas (por exemplo, acurácia, precisão, erro quadrático médio

etc.). Seja $m(E)$ o valor da métrica (m) obtido pelo experimento (E). A métrica de reprodutibilidade é então dada por:

$$\rho(E_1, E_2) = \frac{1}{\|M\|} \sum_{m \in M} \frac{\|m(E_1) - m(E_2)\|}{\|m(E_1)\| + \epsilon},$$

Figura 5 – Métrica de reprodutibilidade
Fonte: Elaborado pelo autor (2026)

Onde: (ϵ) é um valor muito pequeno apenas para evitar divisão por zero. Quanto mais próximo de 0 for o valor de (ρ), mais similares (logo, mais reproduzíveis) são os resultados dos dois experimentos comparados. Em um cenário ideal de reexecução idêntica, todas as métricas seriam iguais, implicando ($m(E_1) = m(E_2)$) para cada ($m \in M$) e, portanto, ($\rho = 0$). Em cenários práticos, pequenas diferenças numéricas podem ocorrer, mas valores de (ρ) baixos (por exemplo, inferiores a 0,01) indicam alta reprodutibilidade, enquanto valores altos significam que algo mudou de forma significativa entre as execuções (por exemplo, bug introduzido no código, alteração acidental nos dados ou não fixação de seed).

Importante: reprodutibilidade não é apenas uma questão acadêmica, mas um requisito para confiabilidade e auditoria em sistemas de aprendizado de máquina. Empresas que operam modelos em produção, frequentemente precisam reproduzir resultados quando auditorias internas ou órgãos reguladores exigem explicações sobre decisões algorítmicas (por exemplo, por que um cliente teve crédito negado). Sem controle de versões, esses requisitos seriam impossíveis de atender. Para visualizar essas ideias na prática, confira a videoaula 2, que ilustra com diagramas como o DVC e outras ferramentas de MLOps ajudam a garantir reprodutibilidade e auditabilidade.

Aspecto	Versionamento de Código (Git)	Versionamento de Dados (DVC)
Volume típico dos artefatos	Arquivos de texto, geralmente pequenos (KB–MB)	Conjuntos de dados possivelmente massivos (GB–TB ou mais)
Unidade de diferença (<i>diff</i>)	Linhas de código (texto), com <i>patches</i> aplicados a arquivos fonte	Arquivos ou diretórios inteiros; diferenças calculadas via hash de conteúdo (MD5)
Armazenamento de versões	Cada versão normalmente armazena cópia completa das alterações no repositório Git	Armazenamento <i>content-addressable</i> com deduplicação: versões compartilham partes idênticas automaticamente
Ferramentas comuns	Git, SVN, Mercurial (controle de versão de código fonte)	DVC, Pachyderm, LakeFS (controle de versão de dados e modelos)

Figura 6 - Comparação entre versionamento de código (Git) e versionamento de dados (DVC)
 Fonte: Elaborado pelo autor (2026)

A tabela 1 resume algumas diferenças-chave entre o versionamento tradicional de código e o versionamento de dados de Machine Learning. Em particular, note que a abordagem adotada pelo DVC (e ferramentas similares) foca em um nível de granularidade diferente: não mais diferenças linha-a-linha em arquivos de texto, mas sim hashes de conteúdo para identificar versões únicas de arquivos e diretórios inteiros de dados. Essa mudança de paradigma é necessária porque dados podem ser binários (imagens, áudio, modelos treinados) e potencialmente muito grandes, onde comparar cada byte seria inviável. Em vez disso, a utilização de hashing criptográfico (como MD5 ou SHA-256) fornece uma assinatura única para cada conteúdo, viabilizando comparações rápidas e armazenamento de deltas com alta eficiência.

Além disso, o DVC introduz o conceito de pipelines declarativos e integrados ao versionamento: você pode definir, em um arquivo YAML, as etapas de pré-processamento, treinamento e avaliação do seu modelo, incluindo a dependência de cada etapa em relação a código, dados de entrada e outputs de etapas anteriores. Esse arquivo de pipeline (`dvc.yaml`) também é versionado no Git e o comando `dvc repro` permite reexecutar automaticamente apenas as partes necessárias do pipeline que foram afetadas por alguma mudança – comportamento similar a sistemas de gerenciamento de build em software tradicional (como `makefiles`). Essa capacidade

provê automação reproduzível, garantindo que os resultados sejam sempre consistentes com os inputs especificados.

Por fim, é importante ressaltar que o controle de versões de modelos é um complemento ao versionamento de dados. Ferramentas de MLOps modernas incluem registros de modelos (model registries), onde cada novo modelo treinado (por exemplo, um arquivo pickle do scikit-learn ou um checkpoint de rede neural) é armazenado com identificadores de versão, métricas de performance e metadados (autor, data, hiperparâmetros, versão dos dados utilizados etc.). Alguns sistemas, como o próprio DVC e MLFlow, permitem versionar modelos lado a lado com dados e código, vinculando cada versão de modelo aos exatos dados e configurações de treinamento utilizados.

Essa rastreabilidade fim a fim – do dado de entrada ao modelo final – é o que permite evitar problemas clássicos de replicabilidade e até mesmo gerenciar implantações de múltiplas versões de modelo em produção (por exemplo, realizar rollbacks caso uma versão atualizada tenha desempenho pior que a anterior). Em resumo, ao controlar diligentemente dados e modelos, a Engenharia de Machine Learning alcança um patamar de confiabilidade e governança equiparável ao da engenharia de software tradicional, porém com ferramentas e formalismos projetados para as particularidades dos sistemas de aprendizado de máquina.

MERCADO, CASES E TENDÊNCIAS

A adoção industrial de ferramentas de versionamento de dados e modelos tem crescido à medida que empresas reconhecem o valor estratégico da reprodutibilidade e da colaboração em ML. Uma pesquisa de Paleyes et al. (2022), com 207 profissionais de Machine Learning, em diversos setores, revelou que 43% das organizações já utilizam alguma ferramenta de controle de versão de dados, sendo o DVC a solução mais citada (adotada por 28% dos respondentes). Essa tendência reflete uma maturidade crescente em MLOps – organizações com melhores práticas de versionamento de dados e modelos tendem também a apresentar maior sucesso na escalabilidade e manutenção de sistemas de ML em produção.

Cases de sucesso da indústria mostram benefícios concretos do DVC e ferramentas similares. A empresa de tecnologia fotográfica VSCO implementou o DVC para gerenciar datasets com milhões de imagens usadas no treinamento de modelos de visão computacional. Como resultado, o tempo médio para reproduzir um experimento complexo caiu em 73%, além de uma economia anual de cerca de US\$ 127 mil em custos de armazenamento, graças à deduplicação automática de dados e à redução de esforços manuais de engenharia (KUPRIEIEV et al., 2021).

Já o Banco Inter, uma fintech brasileira, adotou o DVC em sua infraestrutura de MLOps para modelos de crédito e detecção de fraudes. Essa implementação permitiu cumprir requisitos rígidos de rastreabilidade impostos pelo Banco Central do Brasil, registrando exatamente quais versões de dados e modelos foram utilizadas em cada decisão automatizada de concessão de empréstimo. O banco reportou que, após o DVC, o tempo para preparar relatórios de auditoria interna sobre um modelo de crédito em produção caiu de duas semanas para menos de 2 horas, demonstrando um enorme ganho de eficiência e confiança no processo. Outro exemplo de destaque é o uso do DVC no Jet Propulsion Laboratory (NASA), onde a ferramenta ajuda a versionar datasets massivos de sensores de satélite e modelos de aprendizado utilizados em pesquisas espaciais.

A garantia de reprodutibilidade fornecida pelo DVC é crítica para a validação científica de descobertas astronômicas publicadas em journals revisados por pares e

o sistema consegue lidar com petabytes de dados distribuídos em múltiplos data centers. Esses casos evidenciam que o controle de dados e modelos não é apenas uma boa prática acadêmica, mas um diferencial competitivo no mercado de tecnologia. Empresas relatam motivadores como: reprodutibilidade (segurança de recriar resultados e investigar bug fixes), colaboração (trabalho eficiente de times distribuídos sobre os mesmos dados), governança e conformidade (atendendo leis como LGPD/GDPR, que exigem rastreamento de dados pessoais usados em algoritmos), além de eficiência operacional (menos tempo gasto em tarefas manuais de preparação de ambiente, mais confiança para atualizar modelos continuamente).

Tendências emergentes

O ecossistema de MLOps continua evoluindo rapidamente. A empresa Iterative.ai, criadora do DVC, lançou recentemente o DVC Studio, uma plataforma SaaS com interface visual para gerenciamento colaborativo de experimentos, comparação de métricas e visualização de pipelines, complementando o uso da linha de comando do DVC. Outra frente em destaque é a adoção em setores altamente regulados (como finanças, saúde e governo), onde o versionamento rigoroso de dados e modelos atende a demandas de compliance e auditoria – por exemplo, para explicar previsões de modelos de crédito ou diagnóstico médico a partir dos dados originais utilizados.

Simultaneamente, observamos iniciativas acadêmicas e industriais para padronizar a documentação e o versionamento em ML: o NIST (Instituto Nacional de Padrões e Tecnologia dos EUA) tem proposto frameworks como o MLTE (Machine Learning Test & Evaluation) para sistematizar testes e documentação de modelos; pesquisadores do Google e parceiros sugeriram as “Model Cards” – cartões de modelo que resumem informações-chave de modelos preditivos, como seus dados de treinamento, desempenho e limitações (MITCHELL et al., 2019); já Gebru et al. (2021) propuseram as “Datasheets for Datasets”, planilhas padronizadas que documentam origem, composição e condições de uso de conjuntos de dados. O objetivo comum dessas tendências é tornar o desenvolvimento de ML mais transparente, confiável e gerenciável.

Ferramentas de versionamento como o DVC se alinham a esses esforços, fornecendo a infraestrutura necessária para implementar tais práticas: por exemplo, um dataset versionado pelo DVC pode acompanhar metadados exigidos por um Datasheet, enquanto modelos versionados podem ser complementados com Model Cards informando contexto e performance. Em suma, o controle de dados e modelos é hoje reconhecido não apenas como uma conveniência técnica, mas como um pilar essencial da estratégia de IA nas organizações, habilitando inovações como AutoML com rastreabilidade (onde pipelines automáticas geram modelos acompanhados de histórico e documentação) e garantindo que a inteligência artificial seja desenvolvida de forma ética, auditável e sustentável no longo prazo.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, consolidamos os fundamentos e a prática do controle de versões de dados e modelos em Machine Learning. Apresentamos inicialmente as diferenças cruciais entre o versionamento de código tradicional e o versionamento de grandes conjuntos de dados, destacando os desafios específicos deste último – como o grande volume de informações e a natureza evolutiva dos dados – que demandam novas ferramentas e abordagens. Exploramos os princípios por trás do Data Version Control (DVC), incluindo sua arquitetura baseada em metadados versionados no Git, cache local de conteúdo e storage remoto para compartilhamento de dados.

Também discutimos conceitos teóricos importantes, como integridade de dados via funções de hash criptográfico (MD5), armazenamento eficiente com deduplicação de conteúdo e formalizamos matematicamente o conceito de reprodutibilidade em experimentos de ML, entendendo como o controle de todas as variáveis (código, dados, parâmetros e seeds aleatórios) permite repetir experimentos de forma determinística. Apresentamos uma métrica quantitativa para avaliar o quão reproduzíveis são duas execuções de um modelo, reforçando a importância de metas de reprodutibilidade próximas de 100% em aplicações de alto impacto.

Na parte prática, demonstramos um fluxo de trabalho completo com DVC, cobrindo desde a configuração inicial do projeto com Git+DVC até o registro automatizado de pipelines de ML. Vimos na prática como adicionar e versionar datasets com `dvc add`, gerar e interpretar arquivos `.dvc` de metadados, configurar e utilizar storages remotos e até mesmo recuperar versões anteriores de conjuntos de dados e modelos usando o histórico do Git em conjunto com o DVC.

Também simulamos um cenário de colaboração multiusuário, demonstrando como membros de um time podem trabalhar simultaneamente em um projeto de ML sem sobrescrever o trabalho uns dos outros, graças ao versionamento consistente dos dados e modelos. Por fim, exploramos casos reais de adoção do DVC e tendências atuais, mostrando que controlar dados e modelos é hoje uma prática indispensável para viabilizar MLOps robusto, permitindo que sistemas de aprendizado de máquina sejam confiáveis, auditáveis e escaláveis.

REFERÊNCIAS

BHARDWAJ, A. et al. **DataHub: Collaborative data science & dataset version management at scale**. In: CIDR 2015 – 7th Biennial Conference on Innovative Data Systems Research. Asilomar, CA, 2015. Disponível em: http://cidrdb.org/cidr2015/Papers/CIDR15_Paper18.pdf. Acesso em: 18 mar. 2026.

BHATTACHERJEE, S. et al. **Principles of dataset versioning: exploring the recreation/storage trade-off**. Proceedings of the VLDB Endowment, v.8, n.12, p. 1346–1357, 2015. DOI: 10.14778/2824032.2824035.

DATA VERSION CONTROL (DVC) – **Documentação Oficial. Versioning Data and Models**. DVC.org. Disponível em: <https://dvc.org/doc/use-cases/versioning-data-and-models>. Acesso em: 18 mar. 2026.

GEBRU, T. et al. **Datasheets for Datasets**. Communications of the ACM, v.64, n.12, p. 86–92, 2021. DOI: 10.1145/3442188.

GOOGLE CLOUD. **MLOps – Continuous delivery and automation pipelines in machine learning**. 2018. Disponível em: <https://cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning>. Acesso em: 18 mar. 2026.

GUNDERSEN, O. E.; KJENSMO, S. **State of the art: Reproducibility in artificial intelligence**. In: Proceedings of the 32nd AAAI Conference on Artificial Intelligence (AAAI). New Orleans, 2018. DOI: 10.1609/aaai.v32i1.11503.

HALEVY, A. et al. **Goods: Organizing Google's Datasets**. In: Proceedings of the 2016 ACM SIGMOD International Conference on Management of Data. San Francisco, 2016. DOI: 10.1145/2882903.2903730.

KUPRIEV, R. **DVC: Data Version Control - Git for Data & Models**. 2021. Disponível em: <https://zenodo.org/records/4892897>. Acesso em: 30 mar. 2026.

MICROSOFT AZURE. **MLOps – Principles and best practices (Azure MLOps Guide)**. 2023. Disponível em: <https://learn.microsoft.com/azure/architecture/example-scenario/mlops/>. Acesso em: 18 mar. 2026.

MITCHELL, M. et al. **Model Cards for Model Reporting**. In: Proceedings of the ACM Conference on Fairness, Accountability, and Transparency (FAccT). Atlanta, 2019. DOI: 10.1145/3287560.3287596.

PALEYES, A.; URMA, R.-G.; LAWRENCE, N. D. **Challenges in deploying machine learning: a survey of case studies**. ACM Computing Surveys, v.55, n.6, art. 123, 2023. DOI: 10.1145/3527153.

SCULLEY, D. et al. **Hidden technical debt in machine learning systems**. In: Proceedings of the 28th International Conference on Neural Information Processing Systems (NeurIPS). Montréal, 2015. Disponível em:

https://papers.nips.cc/paper_files/paper/2015/file/86df7dcfd896fcaf2674f757a2463eba-Paper.pdf. Acesso em: 18 mar. 2026.

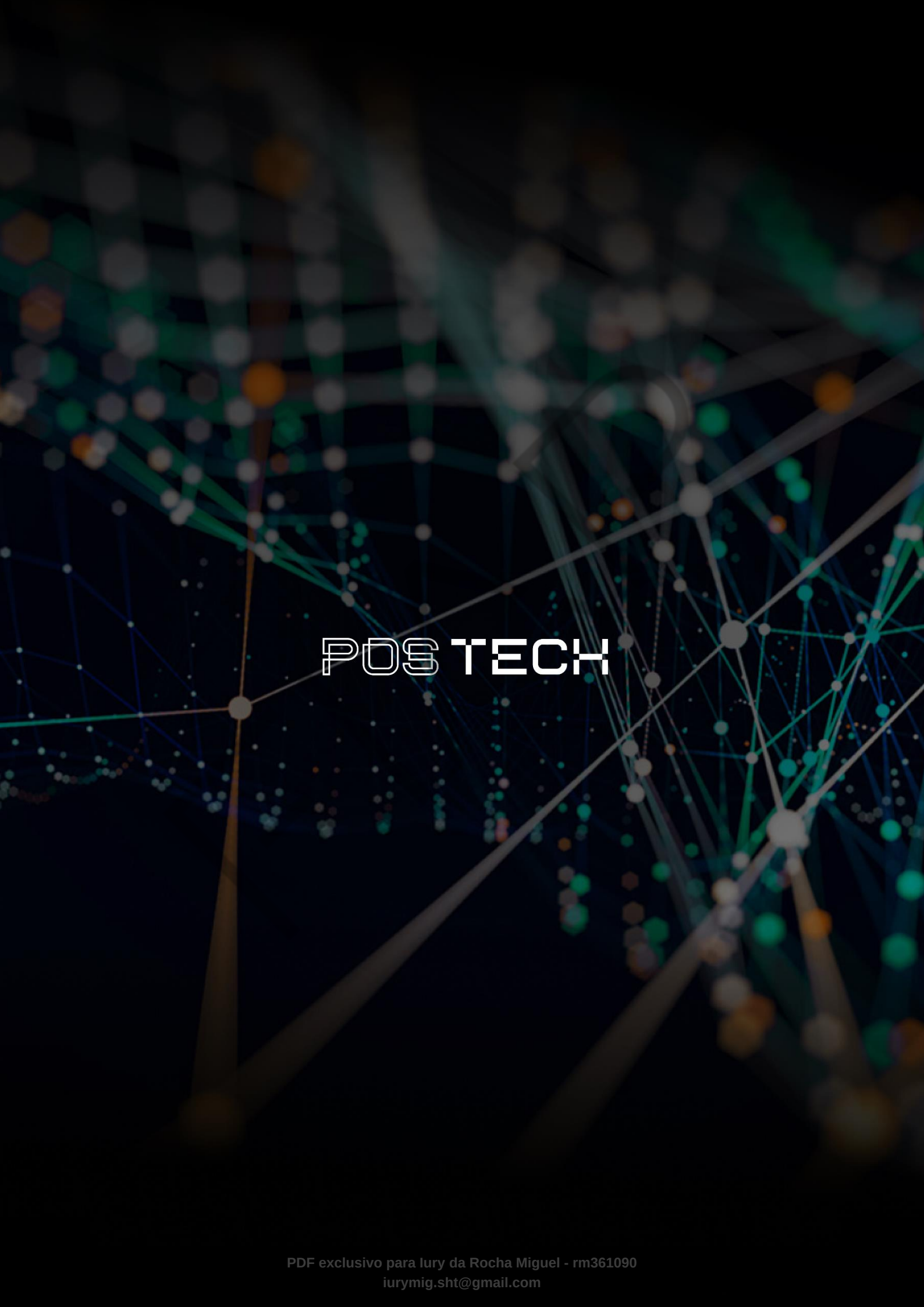
VARTAK, M. et al. ModelDB: **A system for machine learning model management**. In: Proceedings of the 2016 ACM SIGMOD Workshop on Human-In-the-Loop Data Analytics (HILDA). San Francisco, 2016. DOI: 10.1145/2939502.2939516.

EMIS

PALAVRAS-CHAVE

Machine Learning. Versionamento de Modelos. MLOps.

EMENDAS



POSTECH